In section 8.2, we have already established that a brute-force is not practical for applications at scale, and we needed to resort to a brand-new data structure designed to handle multi-dimensional indexing: chapter 9 described k-d trees, a milestone in multidimensional data indexing, a true game changer, which worked perfectly with the example we used in chapter 8 and 9, where we only need to work with 2-D data. The only issue we faced is the fact that our dataset was dynamic and thus insertion/removal would produce an imbalanced tree, but we could re-build the tree in every so often (for instance, after 1% of its elements have been changed because of insertions or removals), and amortize the cost of the operation by running it in a background process (keeping the old version of the tree in the meantime, and either putting insert/delete on hold, or re-applying these operations to the new tree once it's been created and "promoted" to master).

While in this case we can find workarounds, in other applications we wouldn't necessarily be so lucky; there are, in fact, intrinsic limitations that k-d trees can't overcome:

- K-d trees are not self-balancing, so they perform best when they are constructed from a stable set of points, and the number of insert and remove is limited with respect to the total number of elements.
- The *curse of dimensionality*: when we deal with high-dimensional spaces, k-d trees become inefficient, because running time for search is exponential in the dimension of the dataset. For points in the k-dimensional space, when $k \approx 30$ k-d trees can't give any advantage over brute-force search.